# RAJALAKSHMI ENGINEERING COLLEGE

Chennai · Autonomous Institution

*Department of Computer Science and Engineering (Cyber Security)*

## CR23532 — CYBER AND DIGITAL FORENSICS

### Case Study Assignment – II

Consolidated submission — all five unit assignments

**Student**: Nila ELA

**Register No.:** 2116241901074

**Programme:** B.E Computer Science and Engineering (Cyber Security)

**Submitted to:** Dr. Sivakumar Dhandapani, CR23532

**Deadline:** 16 October 2026

### Contents



| Assignment | Scenario / Unit |
|---|---|
| **ASSIGNMENT 2(1)** | Suspected Unauthorized Access to a College Examination Server |
| **ASSIGNMENT 2(2)** | Linking a Suspicious File to a User Using File-System Evidence |
| **ASSIGNMENT 2(3)** | Forensic Imaging and Safekeeping of a Suspect Storage Device |
| **ASSIGNMENT 2(4)** | Recovery of Deleted Evidence from Volume Shadow Copies |
| **ASSIGNMENT 2(5)** | Multi-Source Investigation of a Suspected Cyber Incident |



*This submission documents forensic workflows carried out on self-built practice datasets. All persons, accounts, files and systems are fictional and created for coursework. Screenshot boxes are to be filled with the student's own captured runs following the included lab guide.*

**RAJALAKSHMI ENGINEERING COLLEGE**

Chennai · Autonomous Institution

*Department of Computer Science and Engineering (Cyber Security)*

## CR23532 — CYBER AND DIGITAL FORENSICS

Case Study Assignment – II

**ASSIGNMENT 2(1) · UNIT I**

# Suspected Unauthorized Access to a College Examination Server

**Student**: Nila ELA

**Register No.**: 2116241901074

**Programme**: B.E Computer Science and Engineering (Cyber Security)

**Unit / Topic**Unit I – Digital Forensics Process (Identification, Collection, Examination, Analysis, Presentation)

**Scenario title:** Suspected Unauthorized Access to a College Examination Server

**Submitted to:** Dr. Sivakumar Dhandapani, CR23532

**Deadline:** 16 October 2026

*This report documents a forensic workflow carried out on a self-built practice dataset. All persons, accounts, files and systems are fictional and created for coursework.*

## 1. Case Overview and Assumptions

On 11 October 2026 the Controller of Examinations reported that confidential question-paper files on the departmental server **exam-srv01** had been read and copied outside working hours. This report reconstructs the incident across five sources of digital evidence and documents the forensic process from the first complaint to a presentation-ready finding. The scenario, the server, the user accounts and all files are fictional and were built as a self-contained practice dataset so that every command in this report can be run and verified.

The investigation is bounded by the following stated facts and assumptions:

- The share `exam_share` on exam-srv01 held the folder `QP_2026_Oct` with four question-paper PDFs.
- Samba audit logging and SSH authentication logging were already enabled on the server.
- A workstation, **CSE-LAB3-PC17**, and a SanDisk USB stick recovered from it are available for examination.
- The server clock and the workstation clock are both set to India Standard Time (IST, UTC+05:30); clock accuracy is confirmed in Section 5.
- Where a precise fact is not supplied by the evidence (for example, the physical identity of the person at the keyboard), the report states this as an open question rather than assuming it.

> **Evidence-handling principle followed throughout**
> Every original item is hashed on intake and then set read-only. All examination is performed on forensic copies or on write-protected images, never on the original media. Hash values recorded at intake are re-checked before the report is finalised so that integrity can be demonstrated.

## 2. The Five-Phase Forensic Process for this Case

The investigation follows the standard digital-forensics lifecycle. The flowchart below maps the sequence of activities from the administrator complaint to the presentation of findings; the paragraphs that follow explain each phase as it applies to this case.

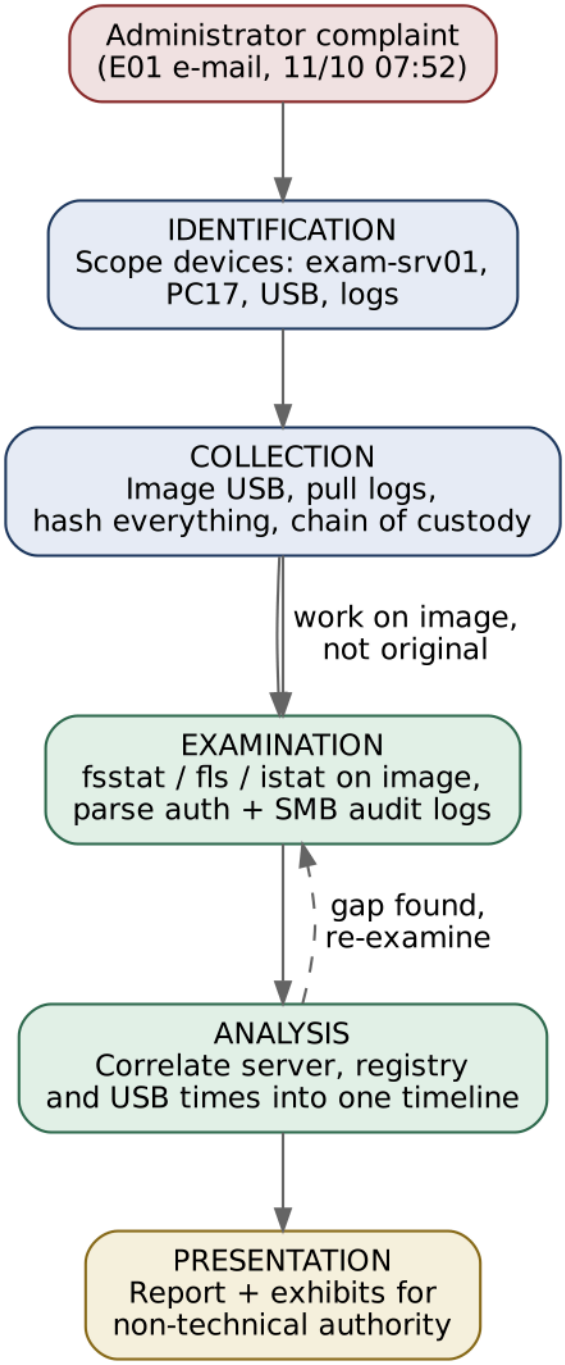



*Figure 1. Investigation flowchart — complaint to presentation, with the five process phases.*

## Identification

The trigger is the complaint e-mail (exhibit E01). From it the team identifies the systems and media in scope: the server exam-srv01, its authentication and Samba audit logs, the suspect workstation PC17, and the USB device recovered from PC17. The investigative question is framed precisely: which account read `QP_2026_Oct` between 01:00 and 02:00 on 11 October, and was the data copied off the server.

## Collection

Log files are exported from the server and the USB device is imaged. Each item is hashed with SHA-256 at the moment of collection and then write-protected, so that later examination cannot alter it. A chain-of-custody entry is opened for every exhibit. Collection prefers the most volatile and most easily altered evidence first; here the live server logs are secured before the physical USB is imaged.

## Examination

Examination is the mechanical extraction of data from the copies: running `fsstat`, `fls` and `istat` against the USB image to list files and recover timestamps, and parsing the authentication and Samba audit logs into readable events. No interpretation happens yet; the aim is a complete, accurate inventory of what each source contains.

## Analysis

Analysis joins the extracted data into a single account of events. The server login time, the registry record of the USB insertion on PC17, the Samba file-read events and the file-creation times on the USB stick are placed on one timeline and cross-checked. Hash comparison establishes whether the files on the USB are identical to the server originals.

## Presentation

The final phase produces this report: a plain-language summary for the Controller of Examinations, supported by exhibits, a timeline and the technical appendix, written so that a non-technical authority can follow the reasoning and a technical reviewer can reproduce it.

## 3. Potential Digital Evidence and its Investigative Value

At least five items of potential digital evidence were identified. The table classifies each by type and by investigative value — that is, how directly it helps answer the investigative question.

| ID | Evidence item | Type | What it can establish | Investigative value |
|---|---|---|---|---|
| E01 | Administrator complaint e-mail | Documentary | Origin, time and scope of the allegation | **Contextual** |
| E02 | Server authentication log | System log | Which account authenticated to the share and when; failed attempts | **High / direct** |
| E03 | Samba file-access audit log | System log | Exact files read from QP_2026_Oct and the read times | **High / direct** |
| E04 | SanDisk USB image (FAT32) | Storage image | Copied question papers and their file-system timestamps | **High / direct** |
| E05 | PC17 USBSTOR registry export | Registry artifact | That this USB was inserted into PC17 and when | **Corroborating** |
| E06 | Server question-paper reference copies | Reference set | Baseline hashes to compare against the USB files | **Corroborating** |

Direct evidence (E02–E04) ties an account and a device to the act of reading and copying the files. Corroborating evidence (E05–E06) strengthens the link without, on its own, proving who sat at the keyboard. Contextual evidence (E01) frames the case but proves nothing about the act.

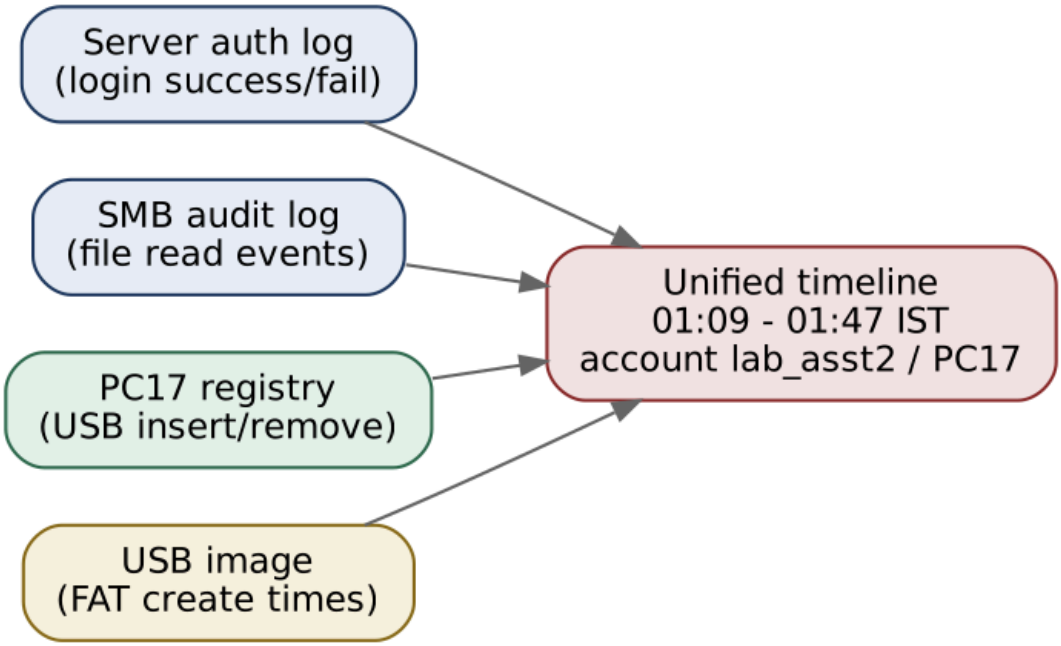



*Figure 2. Four independent sources converge on one account and workstation.*

## 4. Collection and Preservation Precautions

The reliability of the whole case depends on the evidence remaining unchanged from seizure to court. The following precautions were applied and are documented as exhibits.

- **Hash on intake.** Every file is hashed with SHA-256 before anything else is done. The intake hash list is exhibit `intake_hashes.sha256`.
- **Write protection.** The USB device is imaged through a write blocker; the working image is then set read-only (`chmod a-w`) so examination tools cannot modify it.
- **Work on copies.** All `fls` / `istat` / `icat` commands run against the image file, never the physical stick.
- **Chain of custody.** A custody record names who collected each item, when, and every subsequent handover.
- **Documentation.** Commands, tool versions and outputs are logged so a second examiner can reproduce the result.

The command block below produces the intake and write-protection step; Figure 3 shows the pipeline it follows.

## Commands — intake hashing and write protection

```
$ cd ~/cases/A1
$ ls -l --time-style=+'%d/%m/%Y %H:%M' evidence/
$ sha256sum evidence/E0[1-5]* | tee work/intake_hashes.sha256
$ chmod -R a-w evidence && stat -c '%A %n' evidence/E04_usb_sandisk.dd
```

Expected result: a SHA-256 value is printed for each exhibit and the USB image permissions change to `-r--r--r--`, confirming write protection.

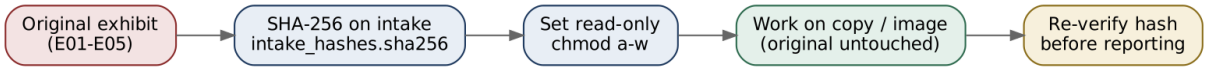



*Figure 3. Evidence intake pipeline: hash on intake, write-protect, then work only on the copy.*

## 5. Building the Timeline of Unauthorized Access

A defensible timeline is assembled from independent sources so that no single artifact has to be trusted alone. Four sources are used: the server authentication log (login success and failures), the Samba audit log (file reads), the PC17 registry (USB insertion and removal) and the USB image itself (file-creation times). Each source carries its own clock, so the examiner records the timezone (IST) and notes that the USB file times reflect the lab machine clock at the moment the practice data was generated, not the case clock.

### Commands — log analysis and USB examination

```
$ grep -E 'smbd' evidence/E02_auth.log            # who authenticated, success and failure
$ awk -F'|' '$5=="openat"{print substr($1,8,8), $3, $NF}' evidence/E03_smb_audit.log
$ fsstat evidence/E04_usb_sandisk.dd | grep -E 'File System Type|Volume ID|Label'
$ fls -r -p -z Asia/Kolkata evidence/E04_usb_sandisk.dd    # list files, recover deleted
$ python3 timeline.py                               # merge all four sources
```

Reference output from the authentication and audit logs (yours will match, since the data is fixed):

```
01:09:27  user [REC\hod_cse]   status [NT_STATUS_WRONG_PASSWORD]  PC17
01:10:02  user [REC\hod_cse]   status [NT_STATUS_WRONG_PASSWORD]  PC17
01:12:44  user [REC\lab_asst2] status [NT_STATUS_OK]              PC17
01:14:05  openat  QP_2026_Oct/CS23511_Compiler_Design_SetA.pdf
01:16:40  openat  QP_2026_Oct/CS23513_Cyber_Forensics_SetB.pdf
```

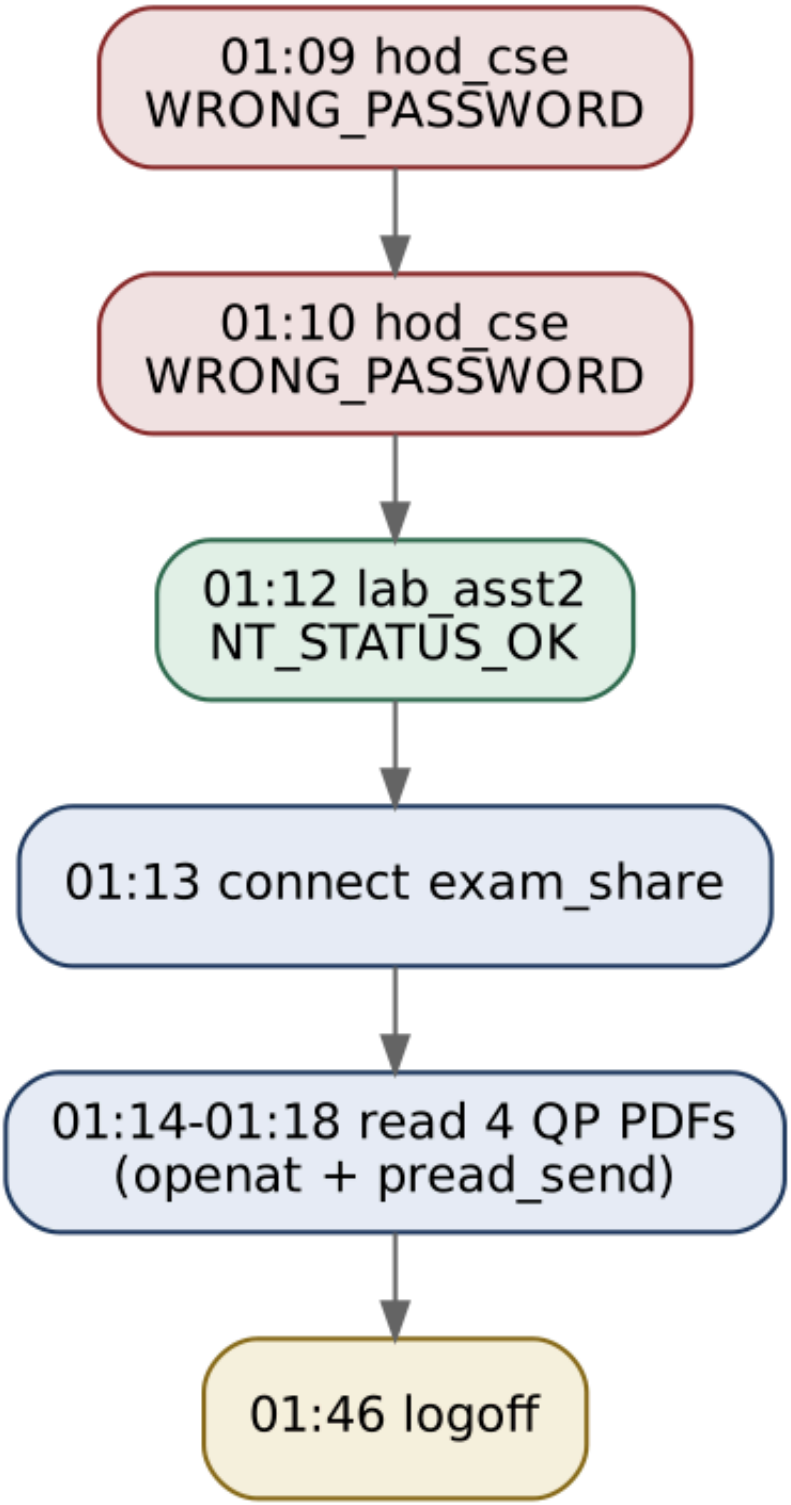



*Figure 4. Authentication and file-access sequence read from the server logs.*

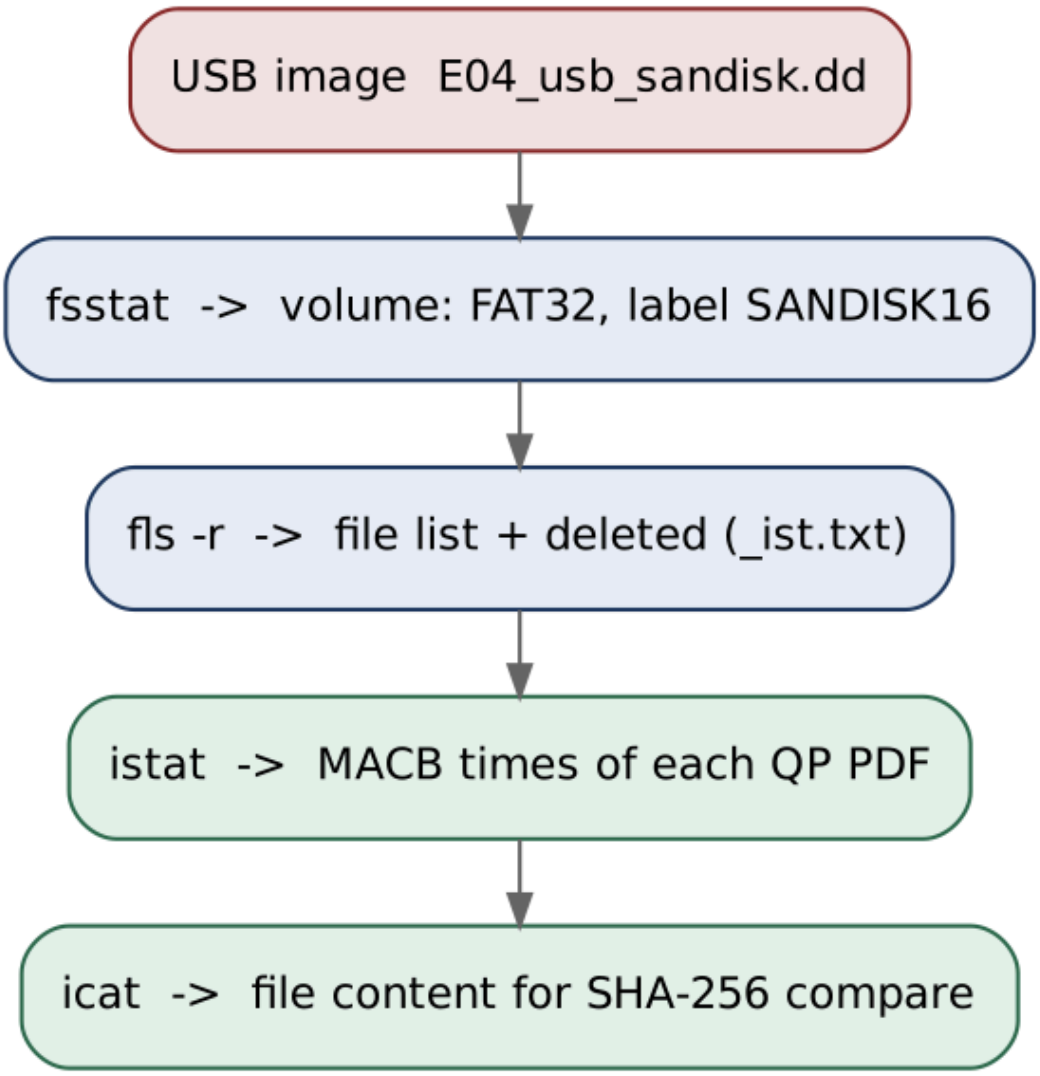



*Figure 5. The Sleuth Kit layer stack applied to the USB image, from volume to file content.*

The merged timeline is the central exhibit. It shows two failed logins under a privileged account, a successful login as `lab_asst2`, the USB being inserted into PC17, four question papers read from the share, matching file-creation times on the USB, and finally the share disconnect and USB removal — all inside a 38-minute window.

| Time (IST) | Source | Event |
|---|---|---|
| 01:09:27 | Server auth | Failed login, account hod_cse, from PC17 |
| 01:10:02 | Server auth | Failed login, account hod_cse, from PC17 |
| 01:11:37 | PC17 registry | SanDisk USB first inserted |
| 01:12:44 | Server auth | **Successful login, account lab_asst2, from PC17** |
| 01:13:21 | Samba audit | Connected to exam_share |
| 01:14:05 – 01:18:02 | Samba audit | Four question-paper PDFs read |
| 01:14 – 01:18 | USB image | **Four matching PDFs created on the USB stick** |

| Time (IST) | Source | Event |
|---|---|---|
| 01:44:30 | Samba audit | Disconnected from exam_share |
| 01:46:58 | Server auth | Logoff, account lab_asst2 |
| 01:47:52 | PC17 registry | SanDisk USB removed |

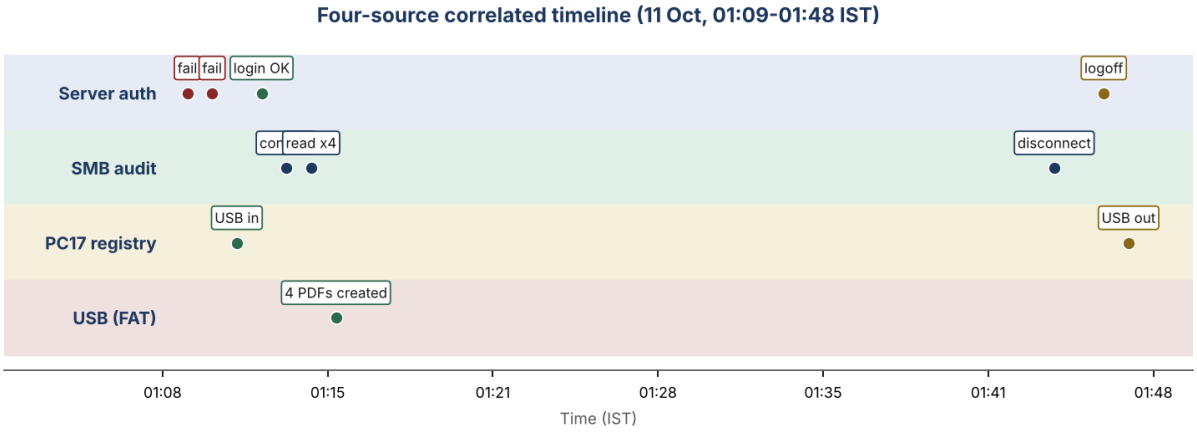



*Figure 6. Four-source correlated timeline of the incident window.*

**Integrity cross-check**

The SHA-256 hash of every PDF carved from the USB image equals the hash of the matching server original. This proves the copied files are bit-for-bit identical to the question papers, not similarly-named decoys.

## 6. Contents of the Final Forensic Report

The report delivered to the Controller of Examinations is structured so that the conclusion is readable without technical knowledge while every claim remains traceable to an exhibit. The recommended structure:

1. Cover and case identifier, examiner name and date.
2. Executive summary — three or four plain sentences: what was asked, what was found, and the confidence level.
3. Scope and authorisation — what was examined and under whose instruction.
4. Evidence inventory — the exhibit table with intake hashes.
5. Methods and tools — named tools and versions, so the work is reproducible.
6. Findings — the timeline and the hash match, each finding tied to an exhibit.
7. Limitations — what the evidence does not establish (see below).
8. Conclusion — the account and device identified, stated at the level the evidence supports.
9. Appendices — full command output, chain-of-custody forms, screenshots.

For a non-technical authority, the executive summary leads with the account name, the window of access, and the fact that the files were copied, and keeps hash values and tool syntax in the appendix. A short annotated timeline figure communicates more than raw log text.

## 7. Legal and Ethical Considerations

Collecting digital evidence in a college environment raises specific obligations:

- **Authorisation.** Examination proceeds only on the written instruction of the competent authority; accessing staff or student accounts without it is itself a violation.
- **Proportionality and privacy.** The examiner looks only at material relevant to the question. Unrelated personal files on a shared workstation are not opened or copied.
- **Account vs person.** The evidence identifies an **account** and a **device**, not conclusively the human who typed. Shared lab credentials, borrowed sessions and unattended logins are realistic alternatives and must be stated.
- **Admissibility (India).** Electronic records require a certificate under Section 63 of the Bharatiya Sakshya Adhiniyam, 2023 (the provision that replaced Section 65B of the Indian Evidence Act); the chain of custody and hash records support this.
- **Neutrality.** The examiner reports what the evidence shows, including anything that weakens the case, and does not advocate for a predetermined outcome.

**Limitations stated in the conclusion**

The evidence shows that the account lab_asst2, from workstation PC17, read four question papers and that identical copies appeared on the recovered USB within the same minutes. It does not, by itself, prove which individual operated the account: the credentials may have been shared or compromised. This limitation is recorded so the finding is not overstated.

**Deliverables in this report:** investigation flowchart (Figure 1), evidence classification table (Section 3), four-source timeline (Section 5), and the forensic-report structure (Section 6). The accompanying lab kit reproduces every command so each figure can be regenerated and verified.

**RAJALAKSHMI ENGINEERING COLLEGE**

Chennai · Autonomous Institution

*Department of Computer Science and Engineering (Cyber Security)*

**CR23532 — CYBER AND DIGITAL FORENSICS**

Case Study Assignment – II

**ASSIGNMENT 2(2) · UNIT II**

# Linking a Suspicious File to a User Using File-System Evidence

**Student:** Nila ELA

**Register No.:** 211624190174

**Programme:** B.E Computer Science and Engineering (Cyber Security)

**Unit / Topic:** Unit II – Computer Forensics: hardware/software environment, file systems, metadata, forensic analysis tooling (ILookIX)

**Scenario title:** Linking a Suspicious File to a User Using File-System Evidence

**Submitted to:** Dr. Sivakumar Dhandapani, CR23532

**Deadline:** 16 October 2026

*This report documents a forensic workflow carried out on a self-built practice dataset. All persons, accounts, files and systems are fictional and created for coursework.*

## 1. Case Overview and Assumptions

A shared office desktop, **ACC-PC04**, is used by several staff of a college accounts section. It holds a suspicious executable, hidden files, encrypted documents, browser artifacts and three local user accounts. The task is to examine the hardware and software environment and determine whether one account — `karthik.m` — was involved in creating and accessing the suspicious files. The syllabus names **ILookIX** as the reference forensic suite; because ILookIX is a licensed Windows product, this report performs the identical workflow with The Sleuth Kit and standard open tools and maps each step to the equivalent ILookIX function, so the reasoning is verifiable on any machine.

Stated facts and assumptions:

- The machine image `ACC-PC04_C.raw` is an NTFS system volume with user profiles `anitha.r`, `karthik.m` and `admin_acc`.
- The suspicious executable is an inert training file: it carries a valid PE header and embedded strings but no working code, so it cannot run.
- Windows authentication logs are not available for this exercise; attribution therefore rests on file-system and application artifacts, and this limit is carried into the conclusion.
- All timestamps are read in IST (UTC+05:30).

> **Safety note on the sample**
> The file FeeSync_Update.exe in this dataset is a harmless placeholder built for the exercise. Real malware is never executed on an examination host; it is analysed statically on an isolated, write-protected image, which is the approach shown here.

The exhibits examined in this report are registered below. Each is hashed on intake and worked on only as a read-only copy.

| Exhibit | Description | Form | Handling |
|---|---|---|---|
| EX-01 | ACC-PC04 system volume | Raw dd image | Hashed, read-only |
| EX-02 | Recovered documents (karthik.m\Documents) | Carved copies | Exported to export/ |
| EX-03 | Chrome History database | SQLite file | Copied via icat |
| EX-04 | Windows Prefetch entry | NTFS file | Listed in place |

## 2. Hardware and Software Components to Document

Before any analysis, the examiner records the environment so that findings can be tied to a specific machine and configuration. The system inventory table is the first deliverable.

| Category | Item | Value / where recorded |
|---|---|---|
| Host | Machine name | ACC-PC04 (NTFS volume label) |
| Storage | Evidence image | ACC-PC04_C.raw, raw dd, 48 MB practice volume |
| Storage | Image hash (SHA-256) | recorded at intake, re-verified before report |
| File system | Type / cluster size | NTFS 3.1, from fsstat |

| Category | Item | Value / where recorded |
|---|---|---|
| OS | Operating system | Windows family (NTFS, Prefetch, registry hives present) |
| Accounts | Local users | anitha.r, karthik.m, admin_acc |
| Applications | Browser | Google Chrome (History SQLite present) |
| Artifacts | Execution evidence | Windows Prefetch (.pf) present |
| Examiner | Tools and versions | The Sleuth Kit 4.12, exiftool, sqlite3 (logged) |

Documenting the hardware and software matters for three reasons: it fixes which file system and timestamp rules apply (NTFS keeps four MACB times per file in two attributes), it tells the examiner which application artifacts will exist (Prefetch, Chrome History), and it lets a reviewer confirm that the right interpretation rules were used.

## Commands — system and volume documentation

```
$ cd ~/cases/A2
$ img_stat evidence/ACC-PC04_C.raw ; sha256sum evidence/ACC-PC04_C.raw
$ fsstat evidence/ACC-PC04_C.raw | sed -n '1,14p'
$ fls -D evidence/ACC-PC04_C.raw          # top-level directories, then Users
```

Expected result: `fsstat` reports **File System Type: NTFS** and the volume label **ACC-PC04**; `fls -D` lists the three user profiles under Users.

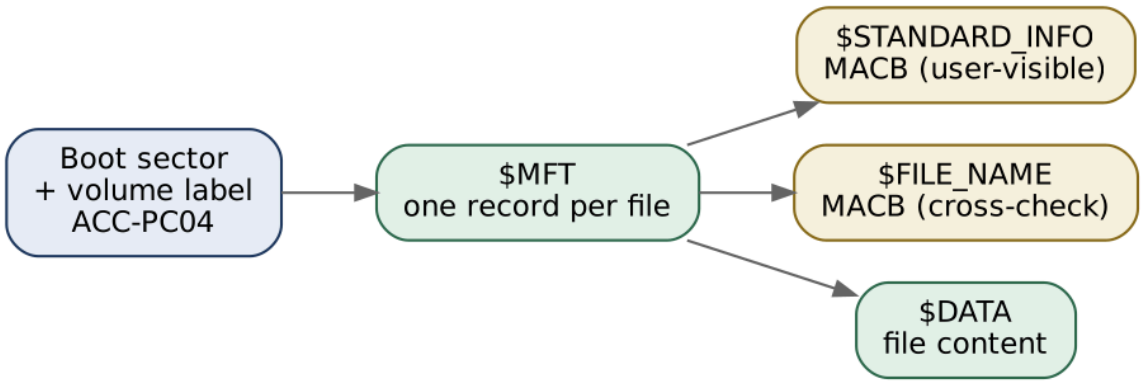



*Figure 1. NTFS per-file structure: the MFT record and the two timestamp attributes used in analysis.*

## 3. How Storage, OS, File Systems and Metadata Locate Evidence

Each layer of the environment narrows the search for relevant data:

- **Storage device.** The raw image is the authoritative source; everything else is derived from it. Imaging the whole device also preserves unallocated space where deleted and hidden data survive.
- **Operating system.** Windows writes predictable artifacts — Prefetch records program execution, the registry records attached USB devices and mounted volumes, and each profile under `Users\` isolates one account's activity.
- **File system.** NTFS stores, for every file, a `$STANDARD_INFORMATION` attribute and a `$FILE_NAME` attribute, each with Created, Modified, MFT-modified and Accessed times. Comparing the two attributes can reveal timestamp tampering, because many anti-forensic tools change only `$STANDARD_INFORMATION`.
- **Metadata.** Office documents embed author, last-modified-by and creation time inside the file; these survive copying and renaming and can link a document to an account even after it is moved.

Together these layers let the examiner move from a whole disk down to a single file attributable to a single profile.

| Layer | Question it answers | Artifact used here |
|---|---|---|
| Storage | What raw data exists, including deleted? | Full dd image + unallocated space |
| OS | Which activity artifacts are present? | Prefetch, registry, profile folders |
| File system | When was each file created/changed? | NTFS $SI and $FN MACB times |
| Metadata | Who authored the document? | Embedded docx author / last-modified-by |

## 4. Hidden and Encrypted Files, and How to Handle Them

Hidden and encrypted files change the investigation in different ways. A hidden file is fully readable once found — the NTFS Hidden/System attribute bits simply keep it out of ordinary directory listings, and forensic tools ignore those bits. An encrypted file is visible but unreadable without the key; the examiner can still prove it exists, measure its size and time, and demonstrate that it is encrypted, which is often evidentially useful on its own.

Shannon entropy distinguishes an encrypted file from an ordinary compressed one. Both have high entropy, so entropy alone is not proof; the examiner also checks the file signature (magic bytes). An OpenSSL-encrypted file begins with `Salted__`; an Office document begins with the ZIP signature `PK\x03\x04` and is merely compressed, not encrypted.

### Commands — hidden attributes and encryption assessment

```
$ fls -r -p evidence/ACC-PC04_C.raw | grep -E 'karthik.m/(Doc|App.*Crypto/)' | grep -v '\$'
$ istat evidence/ACC-PC04_C.raw <inode>   | grep Flags   # shows Hidden, System
$ tsk_recover -e -d <Documents_inode> evidence/ACC-PC04_C.raw export/
$ python3 entropy.py export/Refund_Request_Oct.docx \
```

```
      export/.cache/ledger_q2.xlsx.enc export/.cache/fee_refunds.7z
```

Reference output of the entropy and signature check:

```
FILE                      BYTES  ENTROPY  MAGIC        ASSESSMENT
Refund_Request_Oct.docx   36690   7.9626  ZIP/OOXML    compressed container, readable
ledger_q2.xlsx.enc        14272   7.9877  OpenSSL enc  encrypted, key required
fee_refunds.7z             6000   7.9701  7-Zip        encrypted, key required
```

**Evidence-handling for encrypted material**

The examiner documents the existence, name, size, location and timestamps of an encrypted file and records that it is encrypted. Attempting to break encryption is out of scope and legally sensitive; the lawful routes are a disclosed password, a key found elsewhere on the system, or a production order. The hidden-attribute and encryption findings are recorded as fact, and the inability to read the plaintext is stated as a limitation.

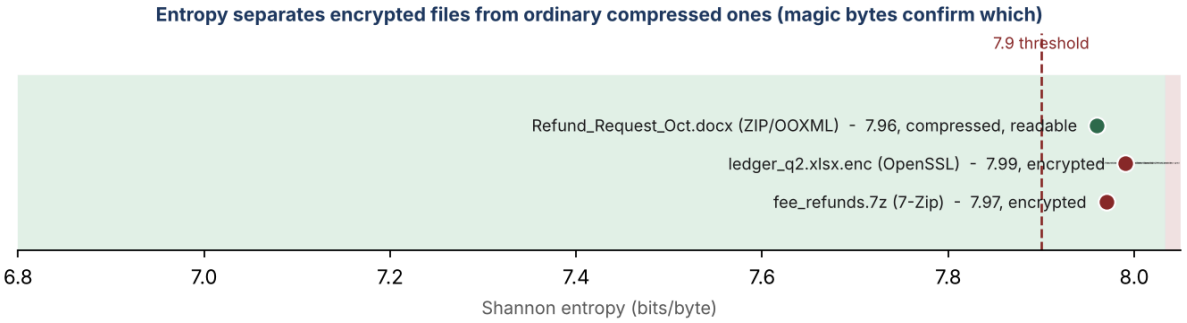



*Figure 2. Entropy of the three files against the 7.9 threshold; the file signature, not entropy alone, separates encrypted from compressed.*

## 5. Evidence-Linkage Matrix

The core deliverable links each artifact to the `karthik.m` account by correlating file names, paths, timestamps, metadata and browser records. The matrix distinguishes what each artifact proves.

| Artifact | Path / location | Links to user via | Key timestamp (IST) | Strength |
|---|---|---|---|---|
| FeeSync_Update.exe | karthik.m\AppData\...\Crypto\ | Inside karthik.m profile; Hidden+System | created 18:41, moved 18:47 | Supporting |
| Refund_Request_Oct.docx | karthik.m\Documents\ | Embedded author = karthik.m | created 09/10 13:24 | **Direct** |
| ledger_q2.xlsx.enc | karthik.m\Documents\.cache\ | In user's hidden folder; encrypted | placed 18:58 | Supporting |
| Chrome History | karthik.m\...\Chrome\Default\ | Download + search tied to profile | download 18:41 | **Direct** |
| Prefetch .pf | Windows\Prefetch\ | Records the .exe executed (machine-wide) | run 18:48 | Contextual |

The browser record is decisive for attribution: the Chrome profile belongs to `karthik.m`, and inside it the download of `FeeSync_Update.exe` and a web search for how to hide a file appear minutes before the file is moved into a hidden folder. The document metadata independently names the same account.

### Commands — metadata and browser history

```
$ exiftool -s -Creator -CreateDate export/Refund_Request_Oct.docx
$ icat evidence/ACC-PC04_C.raw <History_inode> > export/History_karthik.m
$ sqlite3 -header -column export/History_karthik.m \
    "SELECT datetime(start_time/1e6-11644473600,'unixepoch','+05:30'),
            target_path, received_bytes FROM downloads;"
$ sqlite3 export/History_karthik.m \
    "SELECT datetime(v.visit_time/1e6-11644473600,'unixepoch','+05:30'),
            u.url FROM visits v JOIN urls u ON u.id=v.url ORDER BY 1;"
```

Reference output — the download row and the visited URLs:

```
2026-10-09 18:41:02  C:\Users\karthik.m\Downloads\FeeSync_Update.exe  11215
18:36:12  https://mail.rec.example.edu/inbox
18:40:55  https://feesync-update.example.net/download?v=2.1
18:45:31  https://www.google.com/search?q=how+to+hide+file+windows+10
19:02:44  https://feesync-update.example.net/api/v2/status
```

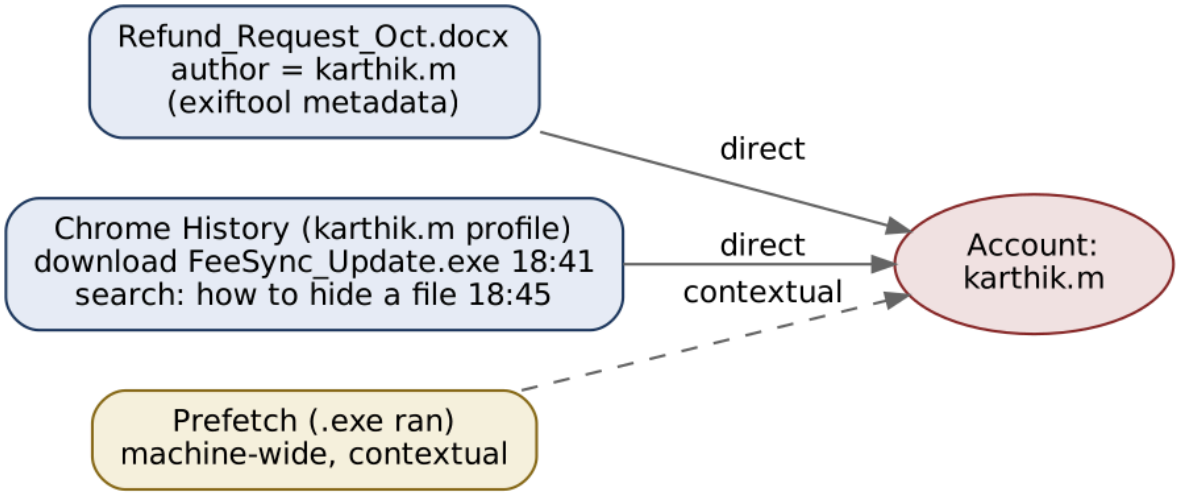



*Figure 3. Which artifacts link directly to the karthik.m account and which only add context.*

## 6. ILookIX-Equivalent Acquisition and Analysis Workflow

The workflow below mirrors the ILookIX process taught in the syllabus. Each row states the ILookIX step and the open-tool equivalent used in this report.

| Step | ILookIX function | Equivalent used here |
|---|---|---|
| 1. Import evidence | Add image to a new case | Open ACC-PC04_C.raw in The Sleuth Kit |
| 2. Verify integrity | Hash verification on import | sha256sum vs intake hash |
| 3. Build file list | Process / index the volume | fls -r -p (full recursive listing) |
| 4. Surface hidden items | Attribute and deleted-file view | istat Flags; fls -d for deleted |
| 5. Metadata review | Property / metadata pane | exiftool on recovered documents |
| 6. Application analysis | Internet history module | sqlite3 on Chrome History |
| 7. Report | Report generator | This document + exhibits |

The figure summarises how the inventory feeds the hidden-file, metadata and browser analyses, which converge on user attribution and then on the stated limitations.

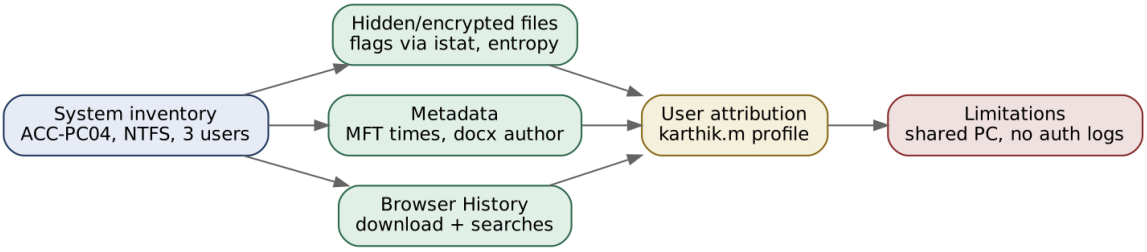



*Figure 4. From system inventory to user attribution, with limitations carried forward.*

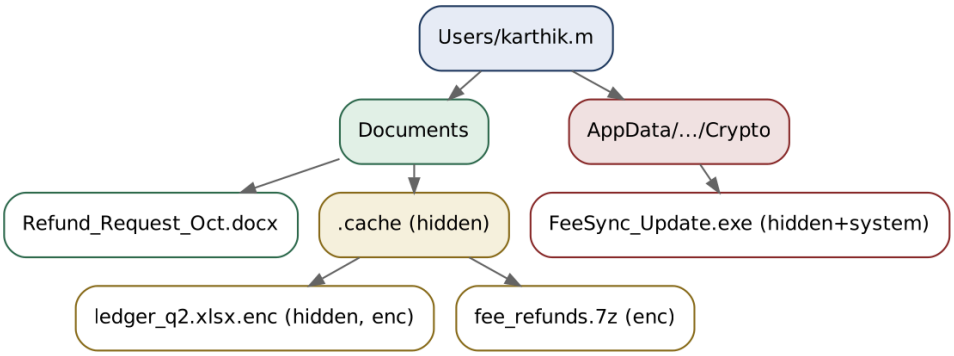



*Figure 5. Layout of the suspicious and hidden files inside the karthik.m profile.*

# 7. Direct vs Contextual Artifacts, and Limitations

## Artifacts that directly link a user to an activity

- **Document metadata** naming `karthik.m` as author and last-modified-by — bound to the content, survives copying.
- **Browser history inside the karthik.m profile** — the download and the "how to hide a file" search are recorded under that specific profile.

## Artifacts that merely provide supporting or contextual information

- **Prefetch** proves the program executed on the machine but is system-wide; it does not say which account launched it.
- **File location inside a profile folder** is suggestive but not conclusive, because another account with administrative rights can write into any profile.
- **The Hidden/System attribute** shows intent to conceal but not who set it.

## Limitations and alternative explanations (at least three)

1. **Shared machine, no auth logs.** Without Windows Security logs, the examiner cannot prove the karthik.m session was operated by the person named karthik.m. Credentials may be shared.
2. **Administrative write-through.** The admin_acc account could place files into the karthik.m profile, so profile location alone is not proof.
3. **Timestamp reliability.** NTFS `$STANDARD_INFORMATION` times can be altered by anti-forensic tools; the `$FILE_NAME` times and application artifacts are cross-checked, but a sophisticated actor could still mislead on timing.
4. **Encrypted content unread.** The contents of the encrypted ledger are unknown, so its relevance is inferred from name and placement, not proven.

**Conclusion**

The file-system and application evidence associates the suspicious executable and the encrypted files with the karthik.m profile, and the document metadata and in-profile browser history provide a direct link to that account. The evidence identifies an account, not conclusively the individual, and the encrypted material remains unread. These limits are stated so the finding is proportionate to the evidence.

**Reproducibility note**

Every result in this report was produced on the self-built image ACC-PC04_C.raw with The Sleuth Kit 4.12, exiftool and sqlite3. The build script, the exact commands and the tool versions are recorded so a second examiner can regenerate the dataset and reach the same file listing, hidden-flag readings, metadata and browser records.

**Deliverables in this report:** system inventory table (Section 2), evidence-linkage matrix (Section 5), ILookIX-equivalent analysis workflow (Section 6, Figure 4), and a conclusion with limitations (Section 7). The accompanying lab kit reproduces every command so each figure can be regenerated and verified.

**RAJALAKSHMI ENGINEERING COLLEGE**

Chennai · Autonomous Institution

*Department of Computer Science and Engineering (Cyber Security)*

**CR23532 — CYBER AND DIGITAL FORENSICS**

Case Study Assignment – II

**ASSIGNMENT 2(3) · UNIT III**

# Forensic Imaging and Safekeeping of a Suspect Storage Device

**Student:** Nila ELA

**Register No.:** 211624190174

**Programme:** B.E Computer Science and Engineering (Cyber Security)

**Unit / Topic:** Unit III – Forensic acquisition: physical imaging, write protection, hashing, evidence containers, data carving, time problems, chain of custody

**Scenario title:** Forensic Imaging and Safekeeping of a Suspect Storage Device

**Submitted to:** Dr. Sivakumar Dhandapani, CR23532

**Deadline**16 October 2026

*This report documents a forensic workflow carried out on a self-built practice dataset. All persons, accounts, files and systems are fictional and created for coursework.*

## 1. Case Overview and Assumptions

A laptop was seized during an investigation into suspected theft of research data. It holds an internal drive and an attached USB stick. The investigation requires that the original media are never modified: the evidence must be physically acquired, imaged into a verified container, carved where the file system is damaged, and kept under an unbroken chain of custody. The syllabus references **IXImager** and the **ASB** container; because both are components of the licensed ILook suite, this report runs the identical workflow with `dc3dd` for imaging and the EnCase **E01** evidence container (the open, widely-admitted equivalent of ASB) and maps each step to its IXImager/ASB counterpart.

Stated facts and assumptions:

- The USB device in this exercise is a 48 MB practice image with an MBR partition table and a FAT32 partition, standing in for the seized Kingston DataTraveler.
- Write protection is enforced in software for the exercise (`blockdev --setro` on a read-only loop device); in casework a hardware write blocker is used and is documented identically.
- The research files, the patent draft and the sensor dataset are fictional training files.
- Seizure time and location are recorded on the chain-of-custody form in Section 8.

> **Core preservation rule**
> The original device is attached only through a write blocker and is read exactly once, to create a verified image. Every later step — listing, carving, analysis — runs against the image or a copy of it. The source hash taken before imaging must equal the image hash taken after; if it does not, the acquisition is repeated.

## 2. Collection and Transport of the Laptop and USB Device

Before imaging, the physical evidence is collected and transported so that it is neither altered nor open to a later challenge that it could have been. Step-by-step procedure:

1. Photograph the laptop and USB in place, with labels and any seals, before touching them.
2. If the laptop is off, leave it off; if on, follow the unit's live-capture procedure before shutdown, because powering down loses memory and may trigger encryption locks.
3. Record make, model and serial numbers of the laptop and the USB device on the seizure form.
4. Place each item in an anti-static evidence bag, seal it with tamper-evident tape, and write the case number, date, time and collector's initials across the seal.
5. Transport in a padded container away from magnetic sources; keep items within sight or locked throughout.
6. Log the handover into the evidence store, starting the chain-of-custody record.

The deliverable for this step is the opened chain-of-custody form (Section 8), begun at the moment of seizure.

## 3. Physical Acquisition and Why Analysis Uses an Image

**Physical acquisition** copies the entire device sector by sector — every byte from the first to the last, including unallocated space, slack space, deleted files and any area outside the mounted partitions. A logical acquisition, by contrast, copies only the live files the file system lists, and would miss exactly the deleted and hidden material an investigation usually needs.

Analysis is performed on the image, not the original, for several reasons: the original is fragile and may fail on repeated reads; opening files on the original updates access times and can alter the evidence; the image can be copied so several examiners work in parallel without touching the source; and the verified hash lets anyone confirm the image still matches what was seized. Working on the original would make every finding challengeable.

## Commands — source identification and write protection

```
$ cd ~/cases/A3
$ LO=$(sudo losetup -r -f --show source/kingston_dt_32g.raw)   # read-only attach
$ sudo blockdev --getro $LO        # must print 1 (read-only)
$ sudo mmls $LO                    # partition table: MBR + FAT32 at sector 2048
$ sudo fsstat -o 2048 $LO | grep -E 'Type|Volume ID|Label'
$ sudo dd if=/dev/zero of=$LO bs=512 count=1   # WRITE TEST: must fail
```

Expected result: `blockdev --getro` prints `1`; `mmls` shows the FAT32 partition starting at sector 2048; the deliberate write test fails with **Operation not permitted**, proving the source is protected.

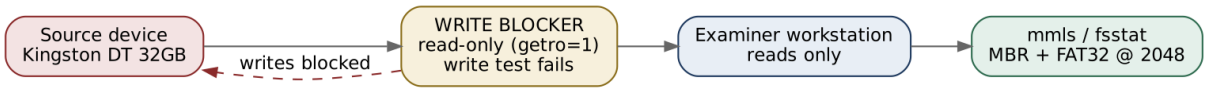



*Figure 1. Write-blocked acquisition path: the examiner reads the source but cannot write to it.*

# 4. Forensic Imaging Workflow with dc3dd (IXImager-equivalent)

The imaging workflow identifies the source, enforces write protection, images with simultaneous hashing, builds a verified container and documents the result. The diagram shows the full chain; the commands follow.

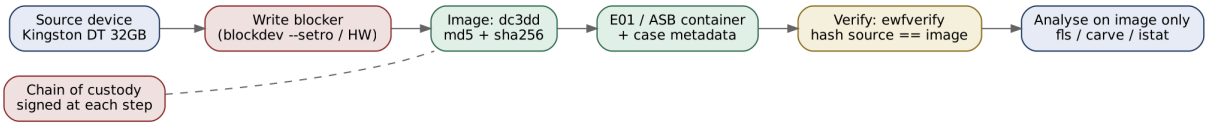



*Figure 2. Imaging workflow — write-blocked source to verified container, under continuous custody.*

| Step | IXImager / ASB | Equivalent in this report |
|---|---|---|
| Identify source | Select physical device | losetup + mmls + fsstat |
| Write-protect | IXImager read-only mode | read-only loop / hardware blocker |
| Image + hash | IXImager acquisition | dc3dd with md5 + sha256 |
| Container | ASB container + metadata | E01 container via ewfacquire |
| Verify | Hash verification | ewfverify re-computes and compares |

## Commands — acquisition, container, verification

```
$ sudo dc3dd if=$LO of=images/EV02_USB_KDT32.dd \
       hash=md5 hash=sha256 log=images/dc3dd.log
$ sudo sha256sum $LO images/EV02_USB_KDT32.dd      # source == image
$ ewfacquire -u -t images/EV02_USB_KDT32 -C REC-CDF-A3-2026-014 \
       -D 'Kingston DT 32GB' -E EV02 -e 'Varsha' -f encase6 -d sha256 \
       images/EV02_USB_KDT32.dd
$ ewfverify -d sha256 images/EV02_USB_KDT32.E01    # expect: SUCCESS
```

Reference result: the SHA-256 printed by `dc3dd` for the source equals the hash of the `.dd` image, and `ewfverify` ends with **SUCCESS**, confirming the container holds an exact, unaltered copy.

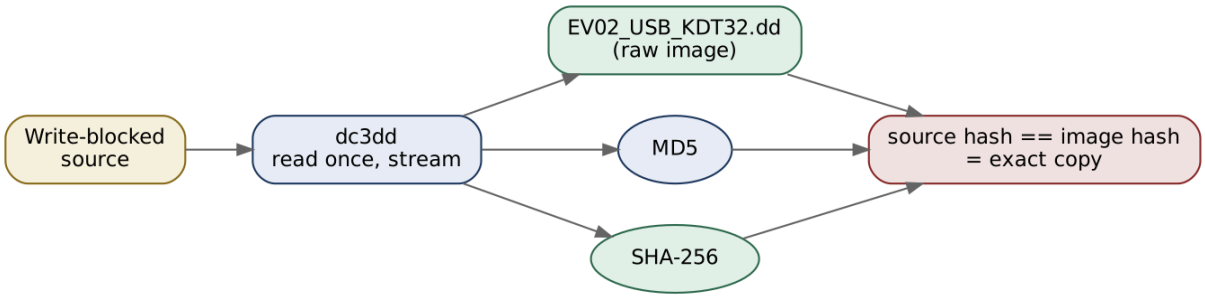



*Figure 3. Acquisition with dc3dd and dual hashing; a source-equals-image hash proves an exact copy.*

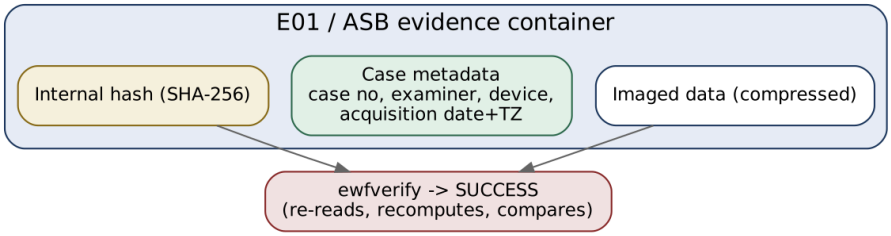



*Figure 4. Contents of the E01 / ASB container and how verification re-proves integrity.*

## 5. Hashing and Integrity Before and After Acquisition

A cryptographic hash is a fixed-length fingerprint of the data; changing a single bit changes the hash completely. Integrity is demonstrated in three recorded points:

- **Before acquisition:** the source device is hashed through the write blocker.
- **During acquisition:** `dc3dd` computes the hash of the data stream as it reads, so the image hash is produced at creation time.
- **After acquisition:** `ewfverify` re-reads the stored container and recomputes the hash.

When all three values agree, the image is proven identical to the seized device and proven unchanged since. SHA-256 is used as the primary hash; MD5 is kept alongside only for cross-checking with older tools, since MD5 is no longer collision-resistant and is not relied on alone.

| Algorithm | Digest size | Status for forensics |
|---|---|---|
| MD5 | 128-bit | Collisions known; secondary cross-check only |
| SHA-1 | 160-bit | Collisions demonstrated; being retired |
| SHA-256 | 256-bit | **Primary integrity hash used in this report** |

### Common acquisition pitfalls and how this workflow avoids them

- **Mounting the source read-write.** Even a read-only intent can update timestamps if the OS auto-mounts the device; the write blocker and the read-only loop device prevent any write reaching the source.
- **Imaging a mounted partition instead of the whole device.** Acquiring the physical device (not a single partition) captures the partition table, unallocated space and any hidden area; `mmls` confirms the full device was imaged.
- **Not hashing at acquisition time.** Hashing only after copying cannot prove the copy matched the source; `dc3dd` hashes the stream as it reads, so the proof is contemporaneous.
- **Silent read errors.** A failing sector can be skipped without notice; `dc3dd` logs bad sectors and pads them, and records the count, so the image's completeness is documented.

# 6. ASB Container, Data Carving and Time Problems

## Role of the ASB / E01 evidence container

An evidence container is more than a copy of the bytes: it stores the image together with case metadata (case number, examiner, description, acquisition date) and an internal hash, and it can be compressed. This binds the identifying information to the evidence so the two cannot be separated, and lets any later tool re-verify integrity from inside the file. Information that should accompany the evidence includes the case and evidence numbers, examiner name, device make/model/serial, acquisition date and time with timezone, the hash values, and the tool and version used.

## Data carving

Data carving recovers files directly from raw bytes by recognising file signatures (headers and footers), without relying on the file system. It is necessary when the file-system metadata is missing or damaged — after a format, a corrupted partition table, or deletion that has cleared directory entries. In this exercise the FAT tables and root directory are wiped on a working copy (never the evidence), after which `fls` can no longer list files, but `foremost` still carves them by signature.

## Commands — carving on a damaged working copy

```
$ bash work/damage.sh          # zero FATs + root dir on a COPY only
$ fls -o 2048 work/EV02_damaged_copy.dd    # file system now unreadable
$ foremost -t jpg,pdf -i work/EV02_damaged_copy.dd -o carve/run1
$ cd carve/run1 && sha256sum jpg/* pdf/*   # compare to known_files.sha256
```

Reference result: `fls` reports the file system can no longer be read, yet `foremost` carves two JPEGs and one PDF whose SHA-256 values match the known research files — proving the carved data is authentic and complete.




*Figure 5. Signature-based carving on the damaged copy, validated by hash against known files.*

## Date and time problems

Timestamps are among the most error-prone evidence. The examiner documents and allows for:

- **Timezone.** FAT stores local time with no timezone; NTFS stores UTC. The examiner records which timezone each tool applied (`fls -z Asia/Kolkata`) so times are not misread by 5.5 hours.
- **Clock accuracy.** The device clock may be wrong, drifting or reset; where possible it is compared to a trusted reference and any skew is recorded.
- **Timestamp meaning.** Created, Modified and Accessed do not all update on every action, and some can be altered deliberately; conclusions rest on several correlated times, not one.

Reference: reading the same file with `-z UTC` and `-z Asia/Kolkata` shows how the displayed time shifts, which is why the applied zone is always stated in the report.

## 7. Chain of Custody for the Seized Laptop

The chain-of-custody record shows continuous, documented control of each exhibit from seizure to examination. Any gap is an opening for challenge. The form below is the deliverable; the student dates and signs the handover rows.

| # | Date / time | Custodian | Action / location | Integrity |
|---|---|---|---|---|
| 1 | 10/10/2026 21:40 | Seizing officer | Seized at Innovation Lab, bagged and sealed | Seal #A-014 |
| 2 | 10/10/2026 22:30 | Evidence store | Logged into secure store | Seal intact |
| 3 | 11/10/2026 __:__ | Examiner (Varsha) | Checked out for imaging | Seal broken, logged |
| 4 | 11/10/2026 __:__ | Examiner (Varsha) | Imaged via write blocker; source returned | SHA-256 recorded |
| 5 | 11/10/2026 __:__ | Evidence store | Original re-sealed and returned to store | New seal #A-021 |
| 6 | __/__/2026 __:__ | Examiner (Varsha) | Analysis on E01 image only | ewfverify SUCCESS |

## 8. Safekeeping of the Image and Evidence

Safekeeping continues after acquisition. The verified image, not just the original device, must be stored so that it stays intact and its integrity can be re-proved at any time.

- **Working copy vs master.** All analysis runs on a working copy; the master image is stored separately and never opened for routine work.
- **Redundant storage.** At least two copies are kept on separate media so a single failure does not lose the evidence; each copy's SHA-256 is recorded.
- **Periodic re-verification.** The stored image is re-hashed on a schedule and before each use, and the value is checked against the acquisition hash to detect silent corruption.
- **Access control.** The evidence store is access-controlled and logged, so every retrieval appears in the chain of custody.
- **Container metadata.** Because the E01 container binds the case metadata and internal hash to the data, a copied container carries its own proof of origin and integrity.

| Safekeeping control | Purpose |
|---|---|
| Read-only master image | Prevents accidental modification during analysis |
| Two copies on separate media | Survives a single storage failure |
| Recorded SHA-256 per copy | Lets any copy be re-verified independently |
| Scheduled re-hash | Detects bit rot or tampering over time |
| Logged, controlled access | Keeps the custody chain unbroken |

**Conclusion**

| Safekeeping control | Purpose |
| --- | --- |
| The device was acquired through a write blocker with dc3dd, sealed into a verified E01 container carrying full case metadata, and confirmed unaltered by matching source, image and verification hashes. Files were recovered by carving from a damaged working copy and validated by hash against known originals, while the original evidence stayed untouched. The chain of custody is continuous and the stored image is kept redundantly and re-verified. The timezone applied to every timestamp is recorded, and clock-accuracy uncertainty is stated rather than assumed away. | |

**Deliverables in this report:** acquisition flowchart (Figure 2), imaging procedure with verification (Sections 4–5), evidence-integrity explanation (Section 5), and the chain-of-custody form (Section 7). The accompanying lab kit reproduces every command so each figure can be regenerated and verified.

**RAJALAKSHMI ENGINEERING COLLEGE**

Chennai · Autonomous Institution

*Department of Computer Science and Engineering (Cyber Security)*

**CR23532 — CYBER AND DIGITAL FORENSICS**

Case Study Assignment – II

**ASSIGNMENT 2(4) · UNIT IV**

# Recovery of Deleted Evidence from Volume Shadow Copies

**Student**: Nila ELA

**Register No.**: 2116241901074

**Programme:** B.E Computer Science and Engineering (Cyber Security)

**Unit / Topic:** Unit IV – Evidence selection and analysis: classification, Event/Cloud/Lead analysis, Volume Shadow Copies, validation, error sources

**Scenario title:** Recovery of Deleted Evidence from Volume Shadow Copies

**Submitted to:** Dr. Sivakumar Dhandapani, CR23532

**Deadline:** 16 October 2026

*This report documents a forensic workflow carried out on a self-built practice dataset. All persons, accounts, files and systems are fictional and created for coursework.*

## 1. Case Overview and Assumptions

A placement-cell employee is suspected of deleting documents and browser records after unauthorized activity on the machine **PLC-PC02**. A forensic image of the Windows system is available and contains Volume Shadow Copies. The task is to select relevant evidence, remove irrelevant data, analyse the artifacts, recover earlier versions from the shadow copy, validate the findings and reconstruct the sequence of events.

Stated facts and assumptions:

- A Volume Shadow Copy (VSS) is a Windows feature that keeps point-in-time snapshots of a volume, letting earlier versions of changed or deleted files be recovered.
- Because VSS snapshots can only be created by Windows, this exercise uses a point-in-time copy of the image, `PLC-PC02_VSS1_snapshot.raw`, taken before the deletions, to stand in for the shadow copy. The analysis technique — compare the snapshot with the current volume and recover what the current volume lost — is identical to real VSS work.
- An exported Windows event log (CSV) and the Chrome History from both states are available.
- All timestamps are read in IST (UTC+05:30).

> **Why shadow copies matter here**
> A suspect who deletes files from the live volume often leaves earlier copies untouched inside a shadow snapshot. Comparing the snapshot to the current volume exposes exactly what was removed or changed, and the earlier versions can be recovered and validated.

## 2. Selecting and Categorising Evidence Before Analysis

Detailed analysis is expensive, so evidence is first selected and reduced. The examiner keeps material relevant to the question — the user's documents, the OneDrive sync folder, browser history and the event log — and sets aside operating-system files, application binaries and other noise that cannot answer whether documents were copied and deleted. Selection is documented so nothing is quietly discarded.

The reduction is recorded as a classification scheme, which also structures the later analysis.

## 3. Evidence Classification Scheme

| Class | Examples in this case | Role in the investigation |
|---|---|---|
| User-created files | Offer_Letters_Batch2026.docx, Recruiter_Contacts_Q4.docx | **Primary subject of the suspected exfiltration** |
| System / application files | Windows binaries, Chrome program files | Set aside as noise after selection |
| Potentially suspicious files | Copies placed in OneDrive\Personal sync folder | Possible exfiltration path |
| Deleted / recovered artifacts | The two .docx deleted from the live volume; history rows removed | **Core evidence of concealment** |

| Class | Examples in this case | Role in the investigation |
|---|---|---|
| Supporting / contextual | Event-log entries, VSS service events, logon/logoff | Ties actions to a user and a time |

This scheme does two things: it justifies which data was carried into deep analysis, and it separates evidence that directly shows the act (deleted documents, trimmed history) from evidence that only provides context (service and logon events).

## Commands — selection and deleted-entry listing

```
$ cd ~/cases/A4
$ sha256sum evidence/*.raw          # record both image hashes
$ fls -r -p -d evidence/PLC-PC02_C.raw | grep -v '\$'   # deleted entries only
$ istat -z Asia/Kolkata evidence/PLC-PC02_C.raw <inode> # deleted doc MACB times
```

Reference result: `fls -d` lists the two deleted .docx files in Documents and their copies in OneDrive\Personal, each marked with * (unallocated).

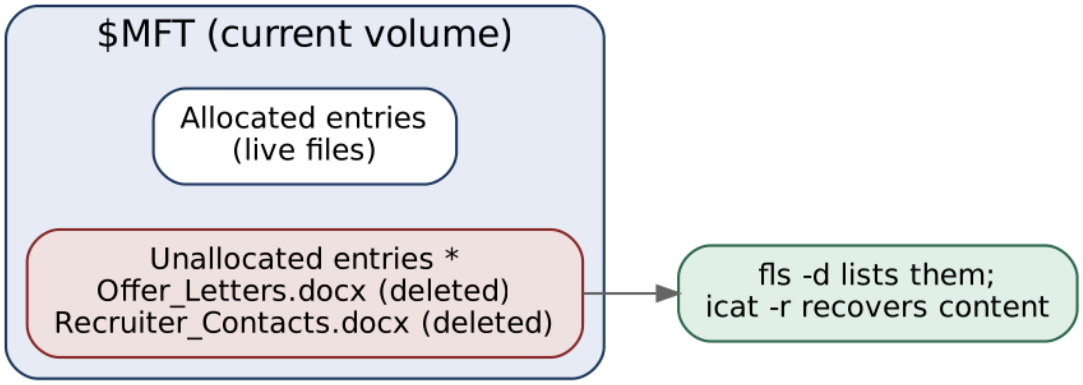



*Figure 1. Deleted files remain as unallocated MFT entries, listed with fls -d and recovered with icat -r.*

## 4. Event Analysis for Sequence Reconstruction

Event Analysis uses the Windows event logs to order what happened and who was present. The exported log for PLC-PC02 records an interactive logon, a write to the offer-letters document, a read of that document into the OneDrive folder, two object-deletion events, a write to the Chrome History file, and finally a System-log-cleared event followed by logoff. Read together these events describe modify → copy-to-sync → delete → trim-history → clear-log, which is a coherent concealment sequence tied to the `deepa.s` account.

Event Analysis is strongest when combined with file-system and application times, because the event log can itself be cleared — as it was here at 17:41. The clearing event is itself evidence.

## 5. Cloud Analysis and Lead Analysis

**Cloud Analysis** examines evidence that an action reached cloud or online services. Here the OneDrive\Personal folder is a local sync point, and the recovered browser history shows a visit to a file-transfer site; both raise the question of whether data left the machine. Cloud Analysis helps answer: was anything uploaded, to which service, when, and is there a server-side record to request.

**Lead Analysis** turns findings into investigative leads: a URL, an account name, an e-mail address or a filename that points to the next source to examine. The questions each answers:

| Analysis | Questions it helps answer |
|---|---|
| Cloud Analysis | Did data leave the device to a cloud service?<br>Which service, account and time?<br>Is there a provider-side log to request under authorisation? |
| Lead Analysis | What is the next artifact or source to pull?<br>Which accounts, URLs or files recur across sources?<br>Where should a production or preservation request be sent? |

## 6. Volume Shadow Copy Analysis

Shadow Copy analysis recovers or compares earlier versions of evidence. The method is to expose the snapshot as a separate volume, list the user's files in both the snapshot and the current volume, and compute the difference. Files present in the snapshot but gone from the current volume are the deleted items; files present in both but different in size or content are the modified items.

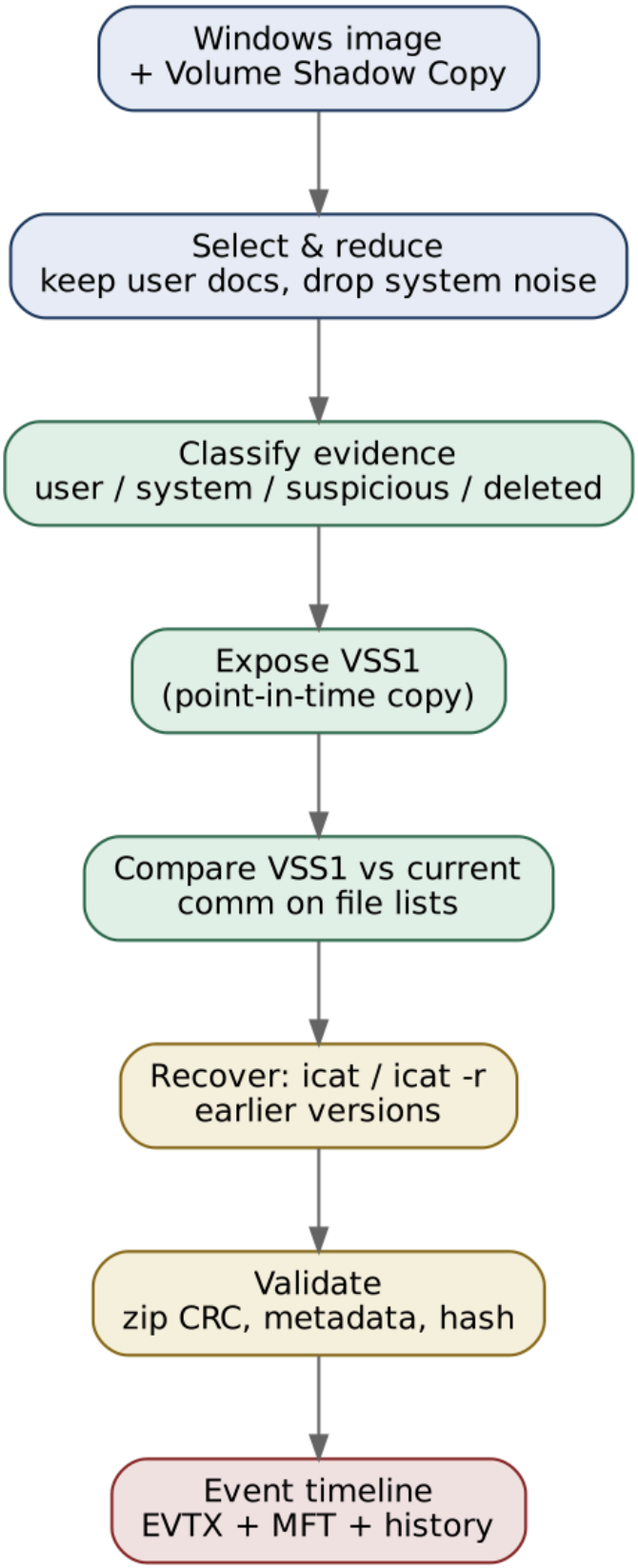



*Figure 2. Shadow-copy recovery workflow — select, classify, compare, recover, validate, timeline.*

## Commands — compare snapshot with current volume

```
$ for i in evidence/PLC-PC02_C.raw evidence/PLC-PC02_VSS1_snapshot.raw; do
    fls -r -p -u $i | grep -E 'deepa.s/(Documents|OneDrive)/.+' \
      | sed 's/^.*:\t//' | sort > work/$(basename $i .raw).lst; done
$ comm -13 work/PLC-PC02_C.lst work/PLC-PC02_VSS1_snapshot.lst  # only in snapshot
```

Reference result: two files appear only in the snapshot — `Offer_Letters_Batch2026.docx` and `Recruiter_Contacts_Q4.docx` — confirming they existed before the deletion and survive in the shadow copy.

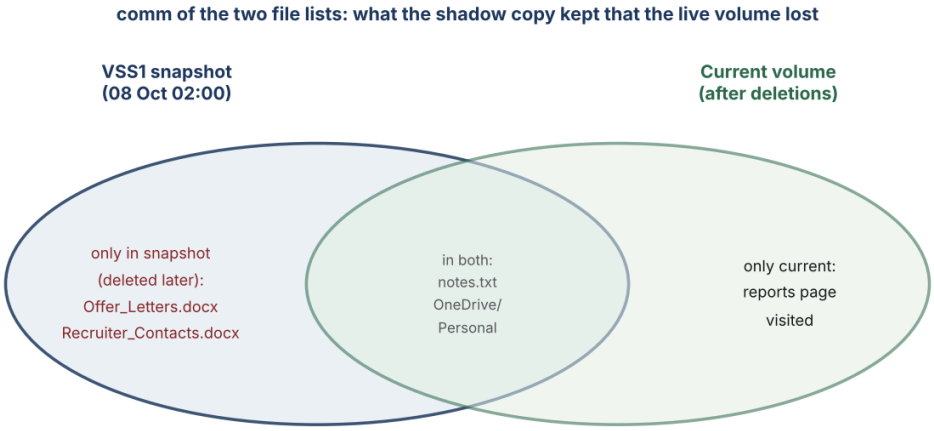



*Figure 3. Comparing the shadow snapshot with the live volume isolates what was deleted.*

## 7. Recovery and Validation of the Artifact

Earlier versions are recovered from the snapshot and, where possible, from unallocated space on the current volume, then validated before being reported.

### Commands — recover and validate

```
$ V=evidence/PLC-PC02_VSS1_snapshot.raw
$ icat $V <inode> > recovered/Offer_Letters_Batch2026_VSS1.docx     # from snapshot
$ icat -r evidence/PLC-PC02_C.raw <deleted_inode> \
      > recovered/Offer_MFT_deleted.docx                            # from current, deleted
$ python3 validate_ooxml.py recovered/*.docx    # zip CRC + parse
$ exiftool -s -T -Creator -RevisionNumber -ModifyDate recovered/*.docx
```

Reference result of validation:

```
Offer_Letters_Batch2026_VSS1.docx  VALID  41 paragraphs, title='Offer Letters Batch 2026'
Offer_MFT_deleted.docx             VALID  47 paragraphs, title='Offer Letters Batch 2026'
Recruiter_Contacts_Q4_VSS1.docx    VALID  31 paragraphs, title='Recruiter Contacts Q4'
```

A recovered artifact is validated before it becomes a finding by: confirming the file structure is intact (the ZIP CRC of the .docx passes), parsing it to confirm it is readable, checking its embedded metadata (author, revision, modified time) for consistency with the case, and hashing it so the recovered object can be re-identified. The comparison is itself a finding: the snapshot holds the earlier 41-paragraph version while the deleted copy on the live volume is the later 47-paragraph revision — evidence that the document was edited, then deleted.

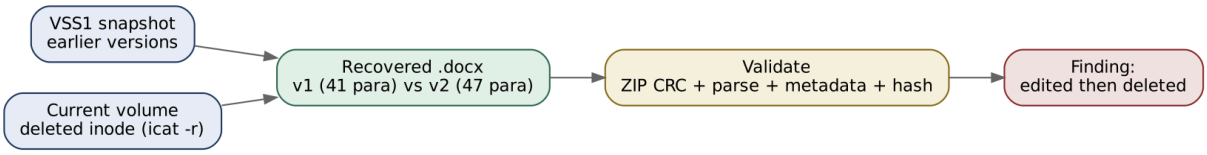



*Figure 4. Recovery from two sources and the validation that turns a recovered file into a finding.*

### Validation checklist

Each recovered artifact is passed through the following checklist before it enters the findings. An item that fails is reported as such, not quietly dropped.

| # | Check | Pass condition | This case |
|---|---|---|---|
| 1 | Structural integrity | ZIP/OOXML CRC passes | **Pass (VSS copies)** |
| 2 | Parses as a document | Opens and lists paragraphs | Pass |
| 3 | Metadata consistency | Author/revision match the case | Pass (deepa.s, rev 3) |
| 4 | Source documented | Snapshot or deleted-inode recorded | Pass |
| 5 | Hash recorded | SHA-256 stored for re-identification | Pass |
| 6 | Cross-checked in time | Fits the event timeline | Pass (17:12–17:31) |

## 8. Event Timeline and Sources of Error

The reconstructed timeline combines the event log, the file-system times from both volumes and the browser-history difference into one sequence.

| Time (IST) | Source | Event |
|---|---|---|
| 08/10 02:00 | Event log | Volume Shadow Copy service ran (snapshot taken) |
| 09/10 17:05 | Event log | Interactive logon, account deepa.s |
| 09/10 17:12 | Event + MFT | Offer-letters document modified (v2, 47 paras) |
| 09/10 17:20 | Event + MFT | Document copied into OneDrive\Personal |
| 09/10 17:31 | Event log | **Two documents deleted** |
| 09/10 17:33 | Event log | Chrome History file overwritten (rows trimmed) |
| 09/10 17:41 | Event log | **System event log cleared** |
| 09/10 17:44 | Event log | Logoff |
| (from VSS1) | Browser | wetransfer.com and webmail visits present only in snapshot history |

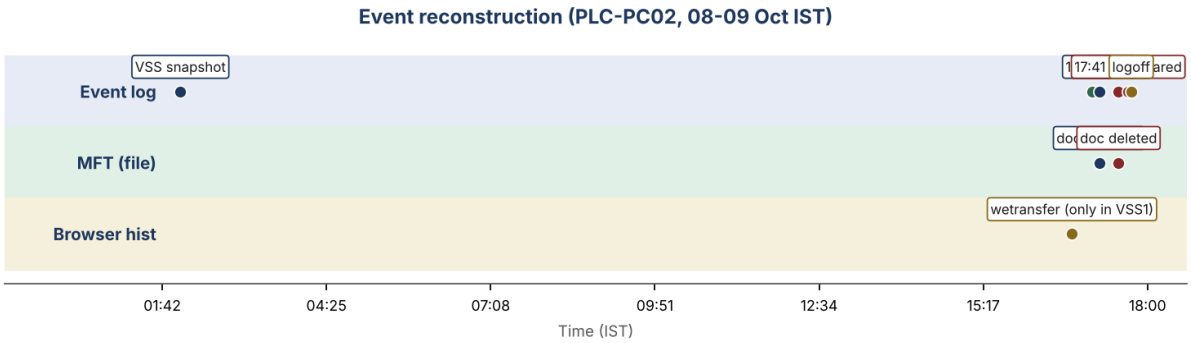



*Figure 5. Event reconstruction across the event log, file system and browser history.*

## Sources of error and alternative explanations

1. **Timestamp interpretation.** NTFS `$STANDARD_INFORMATION` times can be altered; the examiner cross-checks `$FILE_NAME` times and event-log times before relying on any single value.
2. **Snapshot age.** A shadow copy is a snapshot at one moment; a file changed and reverted between snapshots may look unchanged. The snapshot time (08/10 02:00) bounds what it can show.
3. **Deletion vs relocation.** A missing file may have been moved rather than deleted; the OneDrive copy and the event-log deletion events are needed to distinguish the two.
4. **Clock skew and timezone.** If the machine clock was wrong, absolute times shift; the applied timezone (IST) is recorded and any known skew is noted.
5. **Recovered-content partiality.** A file carved from unallocated space may be overwritten in part; the ZIP-CRC and parse checks catch corruption so a damaged file is not reported as sound.

### Conclusion

The shadow copy preserved earlier versions of two documents that were later deleted from the live volume, and the browser-history difference preserved visits that were trimmed from the current history. The event log records modify, copy-to-sync, delete and log-clear actions under the deepa.s account within a 40-minute window. Recovered documents were validated by structure, metadata and hash. The findings identify an account and a sequence; timestamp and snapshot-age limits are stated so the reconstruction is not overstated.

**Deliverables in this report:** evidence classification table (Section 3), event timeline (Section 8), Volume Shadow Copy recovery workflow (Section 6, Figure 2), and a validation checklist (Section 7). The accompanying lab kit reproduces every command so each figure can be regenerated and verified.

**RAJALAKSHMI ENGINEERING COLLEGE**

Chennai · Autonomous Institution

*Department of Computer Science and Engineering (Cyber Security)*

**CR23532 — CYBER AND DIGITAL FORENSICS**

Case Study Assignment – II

**ASSIGNMENT 2(5) · UNIT V**

# Multi-Source Investigation of a Suspected Cyber Incident

**Student**: Nila ELA

**Register No.**: 2116241901074

**Programme:** B.E Computer Science and Engineering (Cyber Security)

**Unit / Topic:** Unit V – Multi-source investigation: Windows and other OS evidence, browser, email/messaging, mobile, Internet/cloud, correlation and challenges

**Scenario title:** Multi-Source Investigation of a Suspected Cyber Incident

**Submitted to:** Dr. Sivakumar Dhandapani, CR23532

**Deadline:** 16 October 2026

*This report documents a forensic workflow carried out on a self-built practice dataset. All persons, accounts, files and systems are fictional and created for coursework.*

# 1. Case Overview and Assumptions

A student account, `arun.p`, is suspected of being used in a cyber incident involving access to the college Learning Management System (LMS) and the export of records. Evidence is spread across a Windows computer, browser artifacts, e-mail, messaging, a mobile phone and cloud or Internet records. The task is to identify what each source can provide, correlate the artifacts into one account of events, and state a conclusion that separates what is proven from what remains open.

Stated facts and assumptions:

- All examination is performed under proper authorisation on lawfully obtained evidence; the exercise uses passive, read-only exports a forensic analyst would receive, not any live access to a service.
- The artifacts are fictional training data. Each source deliberately records time in a different clock — IST, UTC, Unix epoch and WebKit epoch — so the correlation step must normalise them.
- No single source is treated as sufficient on its own; the conclusion rests on agreement across sources.
- The investigative question is whether the arun.p account was used to access the LMS and export a record, and what can and cannot be said about the person behind it.

# 2. Evidence from Windows and Other Operating Systems

A Windows computer offers several artifact classes that show program use and user activity even when the user tried to clean up:

- **Prefetch** records which programs ran, how many times and when last — here a browser, a VPN client and a messaging app.
- **Registry** holds attached-device history, mounted volumes and typed paths.
- **Shellbags** record folders that were opened, including ones since deleted.
- **Event logs** record logon, logoff and account activity.
- **File-system metadata** (NTFS MACB times) orders file creation, modification and access.

Other operating systems provide parallel evidence: Linux keeps shell history, `auth.log` and systemd journal entries; macOS keeps unified logs and FSEvents. The examiner notes which OS is present because it determines which artifacts exist and how their timestamps are stored (for example, FAT local time vs NTFS UTC).

### Commands — normalise every source to one timezone

```
$ cd ~/cases/A5
$ python3 correlate.py     # reads all six sources, converts each clock to IST,
                           # prints one merged timeline
```

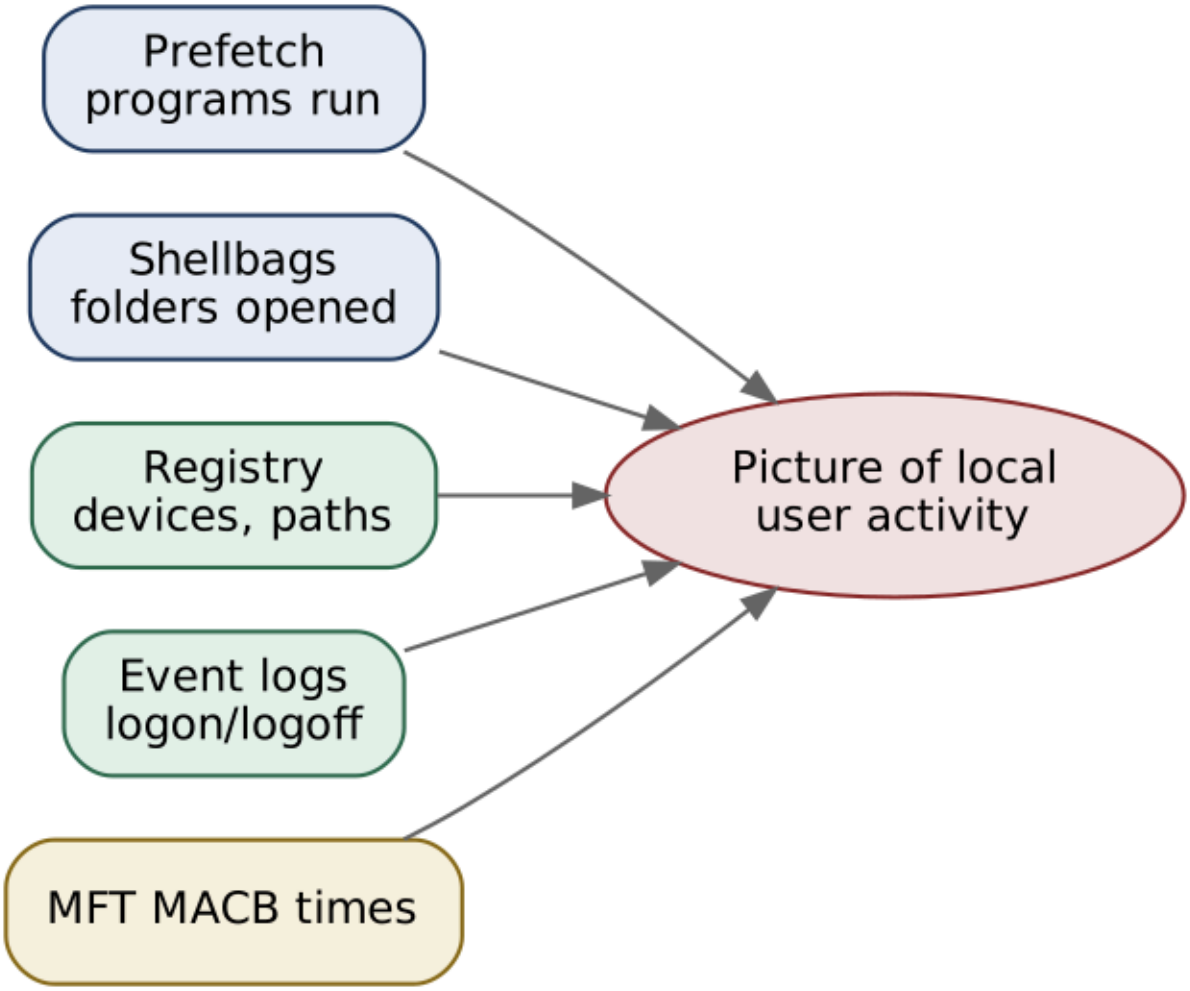



*Figure 1. Windows artifact sources that together describe local user activity.*

## 3. Browser Artifacts and Careful Interpretation

Browser artifacts can reveal visited URLs and their times, searches, downloads, cached pages, cookies and autofill data. In this case the Chrome history shows visits to the LMS login, a course page, the grade report and a webmail service, with times stored in the WebKit epoch (microseconds since 1601 UTC).

Browser evidence must be interpreted carefully for several reasons: a visit does not prove who was at the keyboard; pages can be loaded by redirects, prefetching or background tabs rather than deliberate navigation; times are stored in a non-obvious epoch and are easy to convert wrongly; history can be edited or cleared; and synced profiles can bring in activity from another device signed into the same account. The examiner reports what the history shows and flags these caveats rather than asserting intent.

## 4. Email and Messaging Evidence

E-mail and messaging systems can provide message content, sender and recipient addresses, timestamps, routing headers, attachments and, for webmail, server-side logs. In this case two `.eml` files (a password-change notice and a forwarded record) carry RFC-5322 `Date` headers in UTC, and a messaging export carries message times in the Unix epoch.

These correlate with other artifacts in three ways: the message times sit inside the same window as the browser and server events; the forwarded-record message aligns with the server's record-export event; and the recipient address becomes a lead for further authorised enquiry. E-mail headers also provide an independent clock (the mail server's) to cross-check the workstation clock.

| Source | Clock stored | Example artifact |
|---|---|---|
| Browser history | WebKit epoch (1601 UTC) | LMS login visit |
| E-mail (.eml) | RFC-5322, UTC offset | Password-change notice |
| Messaging export | Unix epoch (seconds) | Chat about the record |
| Mobile SMS db | Unix epoch (milliseconds) | Login OTP message |
| LMS server log | Local time, IST | Record exported event |
| Cloud sign-in log | UTC | SSO sign-in, with location |

## 5. Mobile Phone Forensics and its Challenges

A mobile phone adds SMS and app databases (usually SQLite), call logs, location history, app caches and notification records. In this case an Android `mmssms.db` holds the login OTP and a confirmation message, with times in Unix milliseconds. The forensic considerations and challenges are distinct from a computer:

- **Acquisition difficulty.** Phones are locked, encrypted by default and vary by model; full physical acquisition is often not possible, so logical or file-system extraction is used, capturing less.
- **Encryption and cloud backup.** Much data is encrypted at rest or stored in the vendor cloud, reaching it needs credentials or a lawful request.

- **Volatility.** Phones sync, update and delete continuously; isolating the device (airplane mode or a shielded bag) is needed to prevent remote change or wipe.
- **Timestamp units.** App databases mix seconds and milliseconds; misreading the unit shifts a time by a factor of 1000.
- **App diversity.** Each messaging app stores data differently, so there is no single parser.

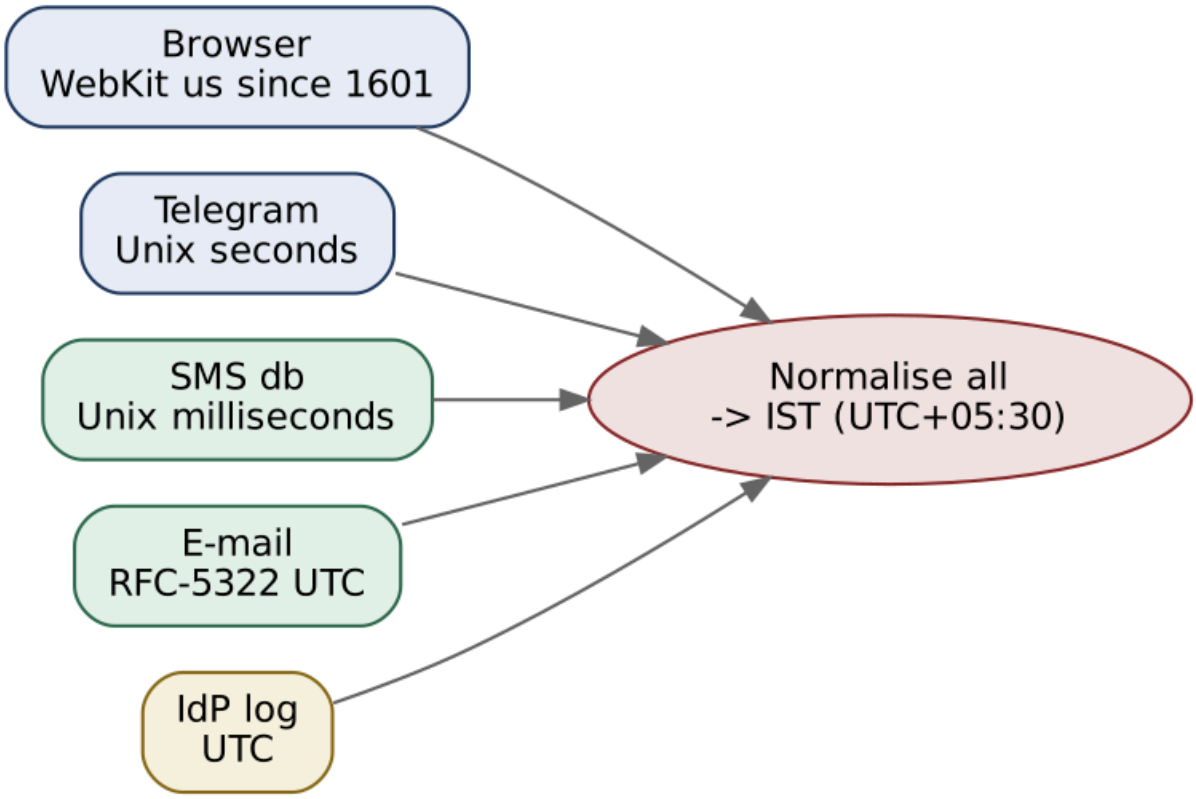



*Figure 2. Each source stores time in a different epoch; all are normalised to IST before correlation.*

## 6. Internet and Cloud Evidence

Internet and cloud sources can be decisive because they hold records the suspect cannot edit locally. Relevant sources here:

- **LMS (application) server logs** — login, page-view and record-export events with the originating IP, in IST.
- **Identity-provider sign-in logs** — single-sign-on records with IP, approximate location and success/failure, in UTC.
- **File-transfer or storage services** — upload records and share links, reached only through the provider under authorisation.
- **ISP / NAT records** — which subscriber held an IP at a given time, needed to resolve an address to a connection.

The cloud sign-in log is particularly useful here: it shows a successful SSO from an overseas IP at the same moment as an interrupted attempt from a local IP, which is a strong lead about how the account was used and a question the local artifacts alone cannot answer.

## 7. Cross-Source Correlation Matrix

The central deliverable normalises every clock to IST and links the artifacts. The matrix shows how five or more hypothetical artifacts from the computer, browser, email/messaging, mobile and cloud line up in time.

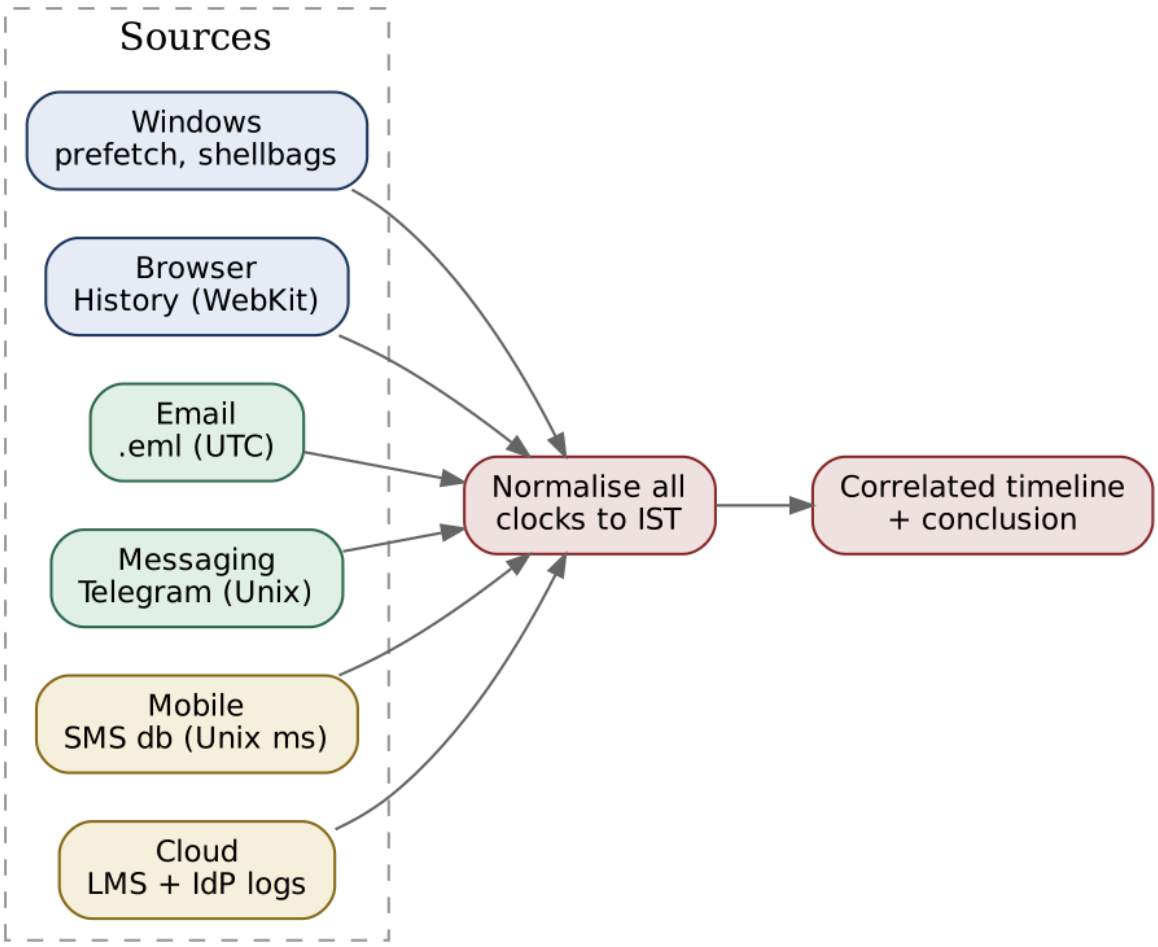



*Figure 3. Six source types normalised to one timezone and correlated into a single timeline.*

| Time (IST) | Source | Artifact / event |
|---|---|---|
| 21:33:38 | Cloud (LMS) | OTP requested for arun.p |
| 21:33:40 | Mobile (SMS) | Login OTP delivered to the phone |
| 21:34:05 | Browser + Cloud | **LMS login visit; SSO sign-in success from overseas IP** |
| 21:41:22 | Browser + LMS log | Grade report viewed |
| 21:47:10 | Cloud (LMS) | **Record exported (PDF)** |
| 21:48–21:52 | Messaging | Chat about the record; says it was forwarded |
| 21:55:07 | E-mail | Record forwarded to an external address |
| 21:55:12 | Mobile (SMS) | Confirmation message: “done, check your mail” |

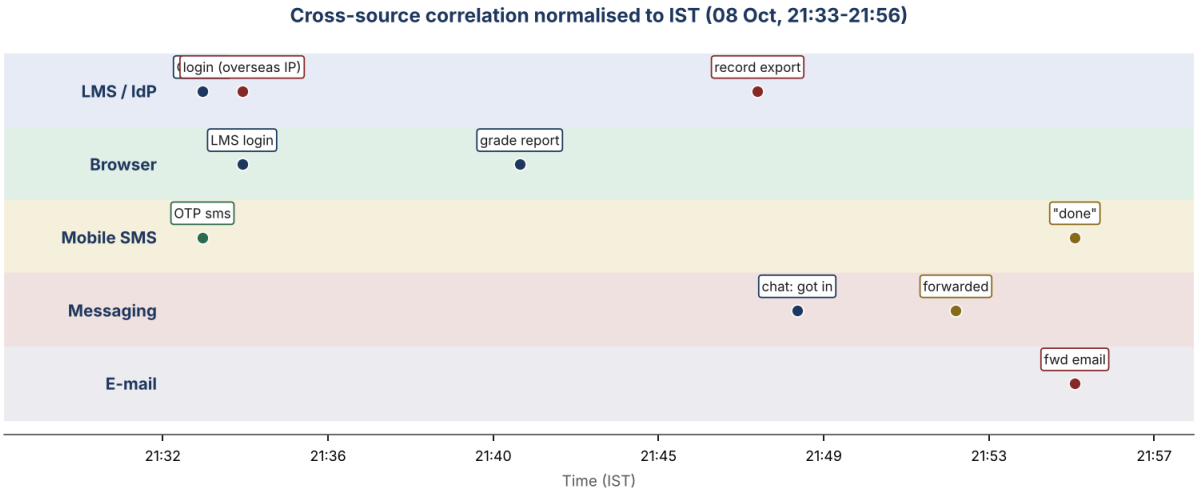



*Figure 4. Cross-source correlation of the incident, every clock normalised to IST.*

## 8. Major Challenges in Multi-Source Digital Forensics

When evidence is distributed across devices and cloud services, at least six challenges arise:

1. **Time synchronisation.** Sources use different clocks and epochs (IST, UTC, Unix seconds, Unix milliseconds, WebKit microseconds); correlation is impossible until all are normalised, and clock skew between devices must be measured.
2. **Volume and relevance.** The combined data is large; selecting the relevant subset without discarding exculpatory material is hard.
3. **Cloud access and jurisdiction.** Key records sit on provider servers in other jurisdictions, reachable only by lawful request, and may be retained only briefly.
4. **Encryption.** Full-disk, app-level and transport encryption block access to content without keys or credentials.
5. **Attribution.** Artifacts identify accounts, devices and IPs, not people; shared credentials, VPNs and NAT break the link between an address and an individual.
6. **Volatility and anti-forensics.** Cloud and mobile data change continuously, and logs or history can be cleared, as happens in several of these labs.
7. **Format diversity and integrity.** Each source has its own format and must be preserved and hashed so the correlated result is defensible.

## 9. Final Investigation Conclusion

The conclusion separates what the correlated evidence proves from what it cannot.

| Category | Finding |
|---|---|
| Direct evidence | Server and identity-provider logs record a successful login and a record export under the arun.p account at 21:34–21:47 IST; the OTP that enabled the login was delivered to the associated phone. |
| Corroborating evidence | Browser history on the computer, the messaging chat, the forwarding e-mail and the SMS confirmation all fall inside the same window and describe the same export-and-forward sequence. |
| Unresolved questions | Whether the person named arun.p operated the account: the SSO succeeded from an overseas IP while a local attempt was interrupted, consistent with either account compromise or deliberate VPN use. The external recipient address and the overseas IP are leads for authorised follow-up. |

**Conclusion**

Correlated across six source types, the evidence shows the arun.p account was used to log in, view and export an LMS record, which was then forwarded externally, all within about twenty minutes.

| Category | Finding |
| --- | --- |
| The account and the sequence are well supported; the identity of the operator is not resolved, because the sign-in originated from an overseas IP while a local attempt failed, and shared or compromised credentials remain a live alternative. These leads are handed on rather than closed. | |

**Deliverables in this report:** multi-source evidence map (Figure 3), cross-source correlation matrix (Section 7), challenge analysis (Section 8), and a final investigative summary separating direct, corroborating and unresolved evidence (Section 9). The accompanying lab kit reproduces every command so each figure can be regenerated and verified.